Bahria University, Islamabad

Department of **Software Engineering**

Artificial Intelligence Lab **(Fall-2026)**

Instructor: **Engr. Saad**

Student: **Abdul Wahab**

Enrollment: **01-131232-007**

Lab Journal: **09**

Date: 15-April-2026

Task No:	**Task Wise Marks**		**Documentation Marks**		**Total Marks (20)**
	Assigned	**Obtained**	**Assigned**	**Obtained**	
1					
2					
3					
4					
5					

Comments:

Signature

Lab 08: File Handling & Regular Expressions

Lab Learning Objectives

- Read data from text files using Python.
- Create and write new files programmatically.
- Read CSV files using both built-in Python and pandas.
- Apply regular expressions to validate or search text patterns.
- Use file handling and regex together in a simple real-world problem.

File Handling:

Python has several functions for creating, reading, updating, and deleting files.

The key function for working with files in Python is the open () function.
The open () function takes two parameters: filename, and mode.

Exercise:

Case Study: You are working as a junior software engineer in a university student support office. You receive a CSV file containing student names, marks, and email addresses. Your task is to automate basic checking.

Problem	Student Task
Some students scored poorly	Identify students having marks less than 12
Email data may be wrong	Use regex to validate the email format
A report is needed	Generate a report.txt file with results
Manager wants proof	Show output in terminal and include screenshots in the manual

Expected deliveries from students:

- Python file containing the final solution
- Terminal screenshot of program output
- Generated report.txt file
- Short explanation of logic used

GitHub Repo Link: https://github.com/Wahab440/AI_Lab_09

Code:

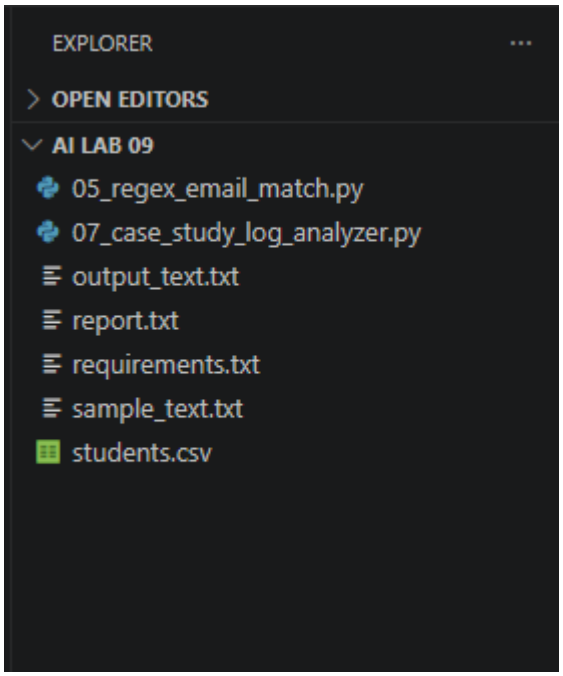

Figure 1: Project Structure

05_regex_email_match.py

05_regex_email_match.py > ...

```python
import re

emails = [
    "student1@university.edu",
    "student.two@uni.edu",
    "wrong-email.com",
    "@nouser.edu",
    "name@domain",
]

email_pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"

for email in emails:
    if re.fullmatch(email_pattern, email):
        print(f"VALID   -> {email}")
    else:
        print(f"INVALID -> {email}")
```

05_regex_email_match.py | 07_case_study_log_analyzer.py | output_text.txt

07_case_study_log_analyzer.py > ...

```python
import csv
from pathlib import Path

# Paths
base_path = Path(__file__).parent
input_csv = base_path / "students.csv"
report_file = base_path / "report.txt"

underperforming_students = []
invalid_email_students = []

# Read data
with open(input_csv, encoding="utf-8") as f:
    for row in csv.DictReader(f):
        name = row["name"].strip()
        marks = int(row["marks"])
        email = row["email"].strip()

        if marks < 12:
            underperforming_students.append((name, marks))

        # Simple email check: must contain '@' and a dot in the domain part.
        if "@" not in email or "." not in email.split("@")[-1]:
            invalid_email_students.append((name, email))

# Build report
report = [
    "Student Support Report",
    "=" * 22,
    "",
    "Rule 1: Underperforming = marks < 12",
    "",
    "Underperforming Students:",
]

if underperforming_students:
    for name, marks in underperforming_students:
        report.append(f"- {name}: {marks}")
else:
    report.append("- None")

report.extend(["", "Invalid Email Records:"])

if invalid_email_students:
    for name, email in invalid_email_students:
        report.append(f"- {name}: {email}")
else:
    report.append("- None")
```

```python
else:
    report.append("- None")

# Write report
report_file.write_text("\n".join(report) + "\n", encoding="utf-8")

# Summary
print("Analysis completed successfully.\n")
print(f"Total underperforming students: {len(underperforming_students)}")
print(f"Total invalid emails: {len(invalid_email_students)}")
print(f"Report: {report_file.name}")
```

05_regex_email_match.py | 07_case_study_log_analyzer.py | report.txt

report.txt

```
Student Support Report
======================

Rule 1: Underperforming = marks < 12

Underperforming Students:
- Ali: 9
- John: 11
- Usman: 7
- Bilal: 10

Invalid Email Records:
- John: john.doe@university
- Bilal: bilal_at_university.edu
```

students.csv | 05_regex_email_match.py | 07_case_study_log_analyzer.py | report.txt

students.csv

```csv
name,marks,email
Ali,9,ali.khan@university.edu
Sara,15,sara_ahmed@university.edu
John,11,john.doe@university
Fatima,18,fatima.zahra@university.edu
Usman,7,usman12@university.edu
Nida,13,nida+lab@university.edu
Bilal,10,bilal_at_university.edu
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\H11 Campus\Desktop\AI Lab 09> Set-Location "c:\Users\H11 Campus\Desktop\AI Lab 09"; & "C:/Users/H11 Campus/AppData/Local/Programs/Python/Python313/python.exe" "07_case_study_log_analyzer.py"
Analysis completed successfully.

Total underperforming students: 4
Total invalid emails: 2
Report: report.txt
PS C:\Users\H11 Campus\Desktop\AI Lab 09>
```

Conclusion:

The case-study automation works correctly. It reads student data from CSV, identifies students with marks below 12, checks basic email validity, and generates a clear report file. This reduces manual effort, improves accuracy, and provides quick evidence through terminal output and report generation.